Data and Algorithm Analysis

Chapter 11 — Hash Tables

Thang Hoang

Department of Computer Science, Virginia Tech

Outline

Introduction

Hash Tables

Chaining

Hash Functions

Open Addressing

Table of Contents

Introduction

Hash Tables

Chaining

Hash Functions

Open Addressing

Overview

Many applications require a dynamic set that supports **dictionary operations** INSERT, SEARCH, and DELETE/

- Example: a symbol table in a compiler

A hash table is effective for implementing a dictionary.

With a reasonable assumption, expected search time in a hash table is $O(1)$.

Worst-case search time is $\Theta(n)$.

Overview

An ordinary array stores the element whose key is k in position k of the array.

Direct addressing: To find element whose key is k, just look in the kth position of the array.

Direct addressing is only applicable when we can afford to allocate an array with one position for *every* possible key.

- Practical?

Overview

Hash table: an effective way to store a number of keys that are small relative to the number of all possible keys.

- An array whose size is *proportional* to the number of keys to be stored.

Given a key, compute a function of k and use that as the index into the array.

This function is called a **hash function**.

Table of Contents

Introduction

Hash Tables

Chaining

Hash Functions

Open Addressing

Direct-Address Tables

Scenario:

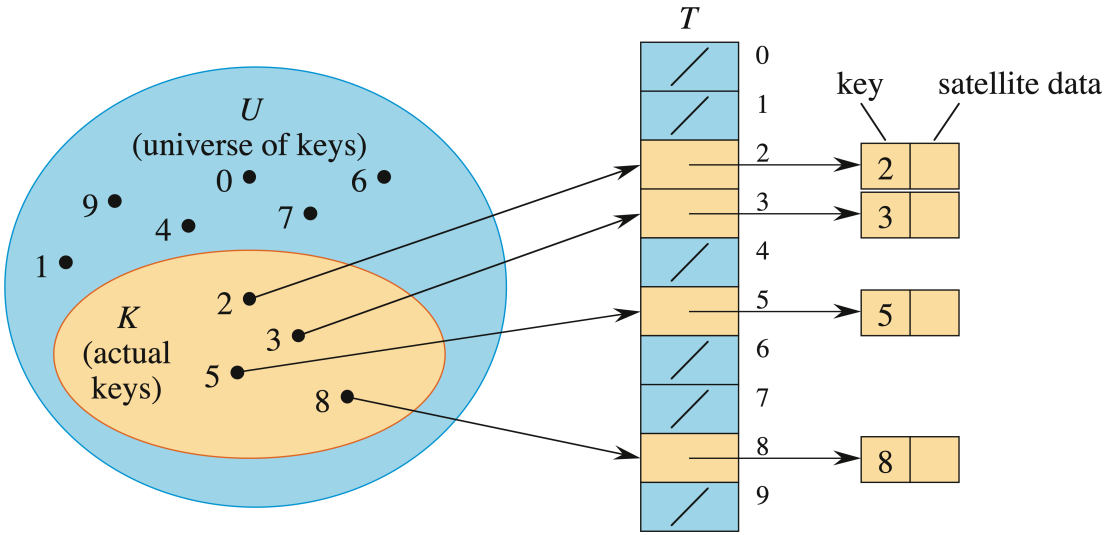
- Maintain a dynamic set
- Each element has a key drawn from $U = \{0, 1, \ldots, m-1\}$, where m is small
- No two elements have the same key

Direct-Address Table is an array $T[0, \ldots, m-1]$:

- Each slot corresponds to a key in U
- $T[k]$ contains a pointer to x if there is an element x with key k
- Otherwise, $T[k]$ is empty (NIL)

Direct-Address Tables

All dictionary operations take $O(1)$-time each



DirectAddressSearch(T, k):

return $T[k]$

DirectAddressInsert(T, k):

$T[x.key] = x$

DirectAddressDelete(T, k):

$T[x.key] = \text{NIL}$

Hash Tables

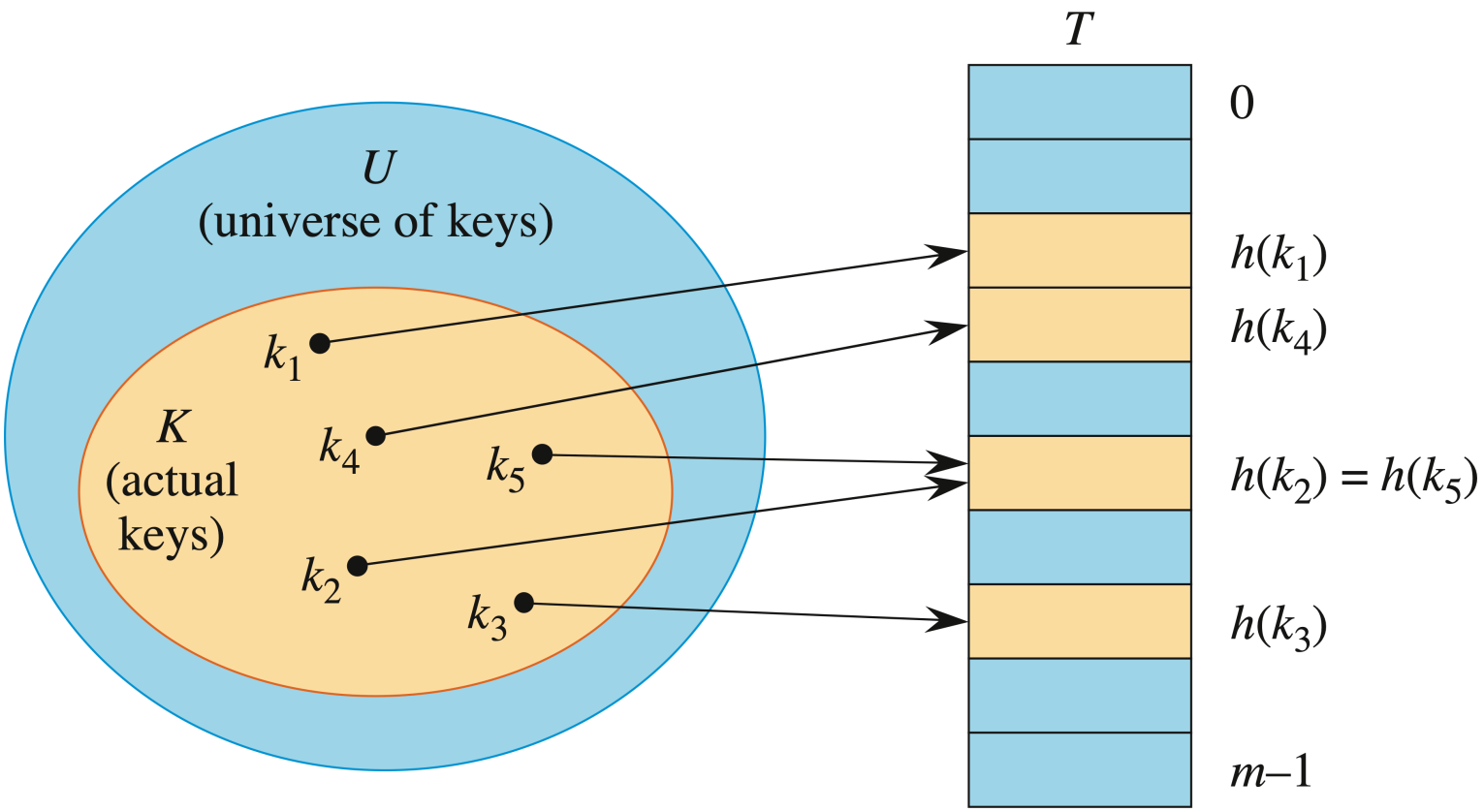
Direct addressing is not practical if the universe $|U|$ is large.

Generally, the set K of keys stored is <u>much smaller</u> than to U.

Main idea of hash tables: Instead of storing x with key k in slot k, use a function h and store x in slot $h(k)$.

- h is called a **hash function**, T is **hash table**.
- $h : U \to \{0, 1, \ldots, m-1\}$.

Hash Tables


T
U
(universe of keys)
K
(actual
keys)
k_1
k_4
k_5
k_2
k_3
0
$h(k_1)$
$h(k_4)$
$h(k_2) = h(k_5)$
$h(k_3)$
$m-1$

Hash Tables

Collision may happen when two or more keys hash to the same slot.

- May happen when $|U| > m$.
- Given a set of keys K, may or may not happen when $|K| \leq m$. Definitely happen when $|K| > m$.

Two methods to address collision: **chaining** and **open addressing**.

Hash Tables

Independent uniform hashing (or **random oracle**)

- $h(k)$ is randomly and independently chosen uniformly from $\{0, 1, \ldots, m-1\}$
- Once $h(k)$ is chosen, each subsequent evaluation of $h(k)$ returns the same result
- An ideal theoretical abstraction. Cannot be reasonably implemented in practice. Why?
- Useful to analyze hashing behavior, and find practical approximations to the ideal.

Table of Contents

Introduction

Hash Tables

Chaining

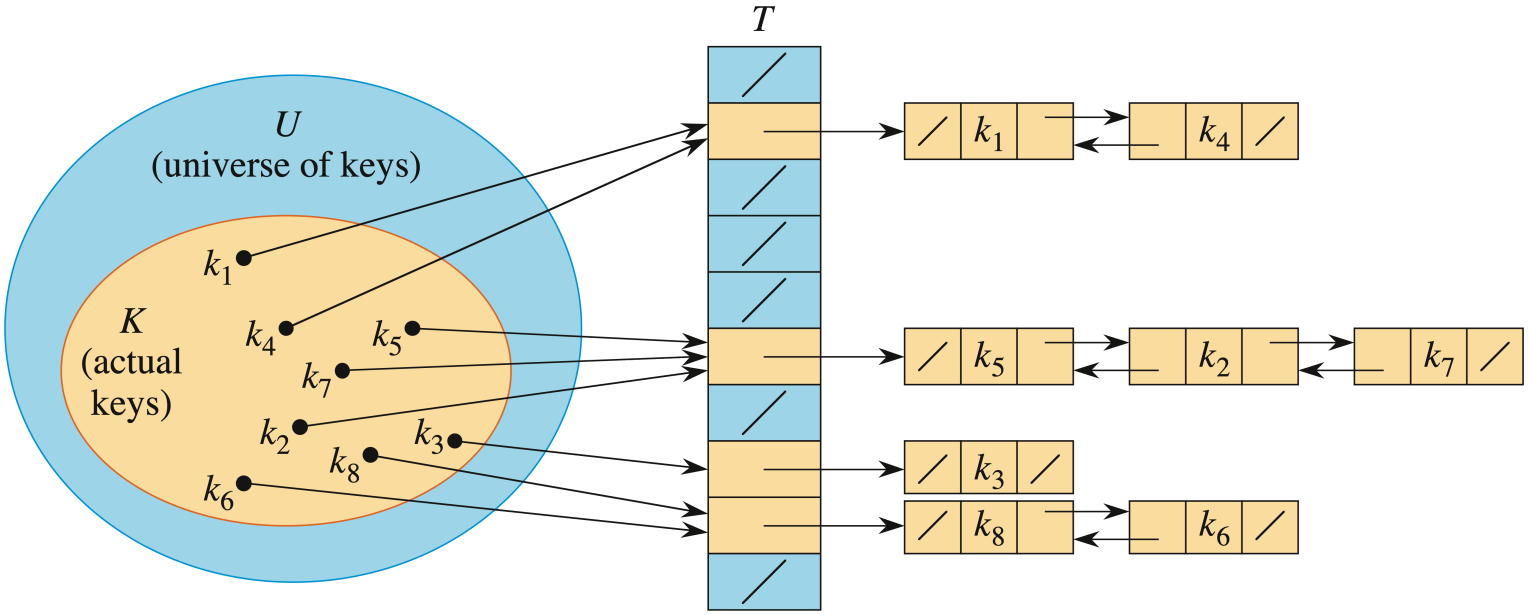
Hash Functions

Open Addressing

Chaining

Non-recursive type of divide-and-conquer:

- Use hash function to divide n elements randomly into m subsets, each with apprx. $|n/m|$ elements.
- Manage each subset independently as a *linked list*.



Chaining

Insertion: $O(1)$ (worst-case running time).

Search: $O(t)$ (worst-case running time), where k is size linked-list at slot $h(k)$.

Delete: $O(1)$ (if implemented with doubly-linked list).

Analysis of Hashing with Chaining

Given a key, how long does it take to find an element with that key, or to determine that there is no element with that key?

Define a load factor $\alpha = n/m$.

- n : # of elements in the table.
- m : # of slots in the table D = # of linked lists.
- α: average number of elements per linked list ($\alpha < 1$, $\alpha = 1$, or $\alpha \geq 1$).

Worst-case search time is $\Theta(n)$ (plus time to compute hash function).

- All n keys hash to the same slot $\Rightarrow$ a single n-sized list.

Average-case search time?

Analysis of Hashing with Chaining

Assume **independent uniform hashing**

- Independent uniform hashing is **universal**: probability that any two distinct keys collide is $1/m$.

Let n_j be the length of list $T[j]$ so that $n = n_0 + n_1 + \ldots, n_{m-1}$.

- $E[n_j] = \alpha = n/m$ for all $j = 0, \ldots, m-1$.

Assume hash function takes $O(1)$ time to compute.

Analysis of hashing with chaining

Let T be a hash table with chaining and independent uniform hashing. Consider two cases:

- T contains no element with key k: unsuccessful search
- T does contain an element with key k: successful search

Theorem

An unsuccessful search on T takes average-case time $\Theta(1 + \alpha)$.

Theorem

An successful search on T takes expected time $\Theta(1 + \alpha)$.

Proof

Analysis of Hashing with Chaining

If $n = O(m)$, then $\alpha = n/m = O(m)/m = O(1)$.

- Searching takes a constant time on average.

Insertion takes $O(1)$ worst-case time, deletion takes O(1) worst-case time (with doubly linked lists).

All dictionary operations take $O(1)$ time on average.

Table of Contents

Introduction

Hash Tables

Chaining

Hash Functions

Open Addressing

What Make a Good Hash Function?

A hash function is **ideal** if it satisfies independent uniform hashing.

- Not practical since the probability distribution that keys are drawn from is unknown.

Can take advantage if the distribution of keys is known.

- Example: If keys are random real numbers independently and uniformly distributed in the half-open interval $[0, 1)$, then can use $h(k) = \lfloor km \rfloor$.

Static Hashing

Single and fixed hash function.

Randomization from the hoped-for distribution of the keys.

Division method

$$h(k) = k \mod m$$

Example: $m = 20$, $k = 91 \Rightarrow h(k) = 11$.

Advantage?

Good choice for m?

Static Hashing

Multiplication method

$$h(k) = \lfloor m(kA \mod 1) \rfloor$$

1. Choose constant A in the range $0 < A < 1$
2. Multiply key k by A
3. Extract the fractional part of kA
4. Multiply the fractional part by m
5. Take the floor of the result

Advantage?

Disadvantage?

Random Hashing

Static hash function is vulnerable to **collision attacks**

- Adversary knows which hash function is used in advance and chooses keys that all hash to the same slot.

Solution: Choose hash function randomly *independent* of keys

- **Universal hashing**

Random Hashing

Definition

Let $\mathcal{H}$ be a finite collection of hash functions that map a universe U of keys into range $\{0, 1, \ldots, m-1\}$.

$\mathcal{H}$ is **universal** if for each pair of keys $k_1, k_2 \in U$, the number of hash functions $h \in H$ for which $h(k_1) = h(k_2)$ is $\leq |\mathcal{H}/m$.

- Probability of a collision between two different keys is $\leq 1/m$ of choosing two slots randomly and independently

Corollary

By chaining and universal hashing, after $n = O(m)$ INSERT operations, the expected time is $\Theta(s)$ to handle any sequence of s INSERT, SEARCH, or DELETE operations.

Achievable Properties of Random Hashing

Uniform: $\Pr[h(k) = q] = 1/m$.

Universal: For any distinct keys k_1, k_2, $\Pr[h(k_1) = h(k_2)] \leq \frac{1}{m}$.

ϵ**-universal**: For any distinct keys k_1, k_2, $\Pr[h(k_1) = h(k_2)] \leq \epsilon$.

d**-independent**: For any distinct keys $k_1, k_2, \ldots, k_d$ and any slots $q_1, q_2, \ldots, q_d$, $\Pr[h(k_i) = q_i] = 1/m^d$.

Designing a universal hash function

Number theory based

- Choose a prime number p large enough so that every possible key is in the set $\{0, 1, \ldots, p-1\}$. Denote $\mathbb{Z}_p = \{0, 1 \ldots, p-1\}$
- Given any $a \in \mathbb{Z}_p \setminus \{0\}$ and any $b \in \mathbb{Z}_p$ define

$$h_{ab}(k) = ((ak + b) \mod p) \mod m$$

- Example: $p = 17, m = 6, a = 3, b = 4$
  $h_{34}(8) = ((3 \cdot 8 + 4) \mod 17) \mod 16 = (28 \mod 17) \mod 6 = 11 \mod 6 = 5$
- Given p and m, the family of hash functions is

$$\mathcal{H} = \{h_{ab} : a \in \mathbb{Z}_p \setminus \{0\} \text{ and } b \in \mathbb{Z}_p\}$$

- $|\mathcal{H}| = p(p-1)$ functions. Each function maps $\mathbb{Z}_p$ to $\mathbb{Z}_m$

Designing a Universal Hash Function

Multiply-shift based

$$\mathcal{H} = \{h_a : a \text{ is odd}, 1 \leq a < n, \text{ and } h_a(k) = ka \mod 2^w \ggg (w - \ell)\}.$$

$\mathcal{H}$ is $2/m$-universal and very efficient in practice.

- Higher collision probability than number-theory based.
- Computation is much faster.

Table of Contents

Introduction

Hash Tables

Chaining

Hash Functions

Open Addressing

Open Addressing

Alternative approach to chaining to handle collision.

- Store all elements in the hash table itself.
- Each slot contains either an element or NIL.
- Load factor always ≤ 1.
- **Probe sequence** to handle collision in INSERTION:
  - Determine element's "first-choice" location. If unoccupied, put it there.
  - Otherwise, determine "second-choice" location. If unoccupied, put the element there.
  - Otherwise, try the "third-choice" location, and so on until an unoccupied location is found.

Open Addressing

SEARCH is the same as INSERTION.

- Look for the first-choice location. If found, return the element.
- Otherwise, look for the second-choice location, and return if found.
- Otherwise, look for the third-choice location, and so on until all occupied locations are scanned, and return NIL if not found.

Open addressing avoids pointers needed for chaining.

Expand the hash table to reduce the collision chance.

Open Addressing

```
HASH-INSERT(T, k)
i = 0
repeat
    q = h(k, i)
    if T[q] == NIL
        T[q] = k
        return q
    else i = i + 1
until i == m
error "hash table overflow"
```

```
HASH-SEARCH(T, k)
i = 0
repeat
    q = h(k, i)
    if T[q] == k
        return q
    i = i + 1
until T[q] == NIL or i == m
return NIL
```

Open Addressing

Deletion cannot simply put NIL into the slot containing the key to be deleted.

- Violate search correctness. Why?

Use a special value DELETED instead of NIL when marking a slot as empty during deletion.

- Search treats DELETED as unmatched key.
- Insertion treats DELETED as empty slot.

Computing Probe Sequence

Ideally, use independent uniform permutation hashing (**uniform hashing**). In practice, approximate it.

Double hashing

- Use auxiliary hash functions h_1, h_2 and probe number i

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m.$$

- Example:
  $m = 13, h_1(k) = k \bmod 13, h_2(k) = 1 + (k \bmod 11)$

Linear probing

- Use an auxiliary hash function h'.

$$h(k, i) = (h'(k) + i) \mod m$$

Analysis of Open-Address Hashing

Assume that

- Hash table never completely fills: $0 \leq n < m \Rightarrow 0 \leq \alpha < 1$;
- Independent uniform permutation hashing;
- No deletion;
- In a successful search, each key is equally likely to be searched for.

Theorem

The expected number of probes in an unsuccessful search is at most $1/(1-\alpha)$.

Analysis of Open-Address Hashing

Corollary

The expected number of probes to insert is at most $1/(1-\alpha)$.

Theorem

The expected number of probes in a successful search is at most $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$.

Proofs on board